AI ASSISTED CODING

Name: P.Swaran Raj **Roll.No:** 2303A52247

Batch: 37 **Assignment -** 21.2

Task 1 – AI-Assisted Pair Programming

Prompt:

"#Generate a simple Python To-Do List Manager, then suggest improvements like error handling and additional features."

Initial Code:

```python
class TodoListManager:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        self.tasks.append({"task": task, "completed": False})
        print(f"Task '{task}' added.")

    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
            return
        print("\n--- To-Do List ---")
        for i, task_item in enumerate(self.tasks):
            status = "[X]" if task_item["completed"] else "[ ]"
            print(f"{i + 1}. {status} {task_item["task"]}")
        print("-----------------")

    def complete_task(self, task_index):
        if 0 <= task_index < len(self.tasks):
            self.tasks[task_index]["completed"] = True
            print(f"Task '{self.tasks[task_index]["task"]}' marked as complete.")
        else:
            print("Invalid task number.")

    def run(self):
        while True:
            print("\n1. Add Task")
            print("2. View Tasks")
            print("3. Complete Task")
            print("4. Exit")

            choice = input("Enter your choice: ")

            if choice == '1':
                task = input("Enter the task: ")
```

```python
            print("4. Exit")

            choice = input("Enter your choice: ")

            if choice == '1':
                task = input("Enter the task: ")
                self.add_task(task)
            elif choice == '2':
                self.view_tasks()
            elif choice == '3':
                self.view_tasks()
                try:
                    task_num = int(input("Enter the task number to complete: ")) - 1
                    self.complete_task(task_num)
                except ValueError:
                    print("Invalid input. Please enter a number.")
            elif choice == '4':
                print("Exiting To-Do List Manager. Goodbye!")
                break
            else:
                print("Invalid choice. Please try again.")


# To run the manager:
todo_manager = TodoListManager()
todo_manager.run()
```

Initial Output:

```
1. Add Task
2. View Tasks
3. Complete Task
4. Exit
Enter your choice: 1
Enter the task: add
Task 'add' added.

1. Add Task
2. View Tasks
3. Complete Task
4. Exit
Enter your choice: 2

--- To-Do List ---
1. [ ] add
------------------

1. Add Task
2. View Tasks
3. Complete Task
4. Exit
Enter your choice: 3

--- To-Do List ---
1. [ ] add
------------------
Enter the task number to complete: 1
Task 'add' marked as complete.

1. Add Task
2. View Tasks
3. Complete Task
4. Exit
Enter your choice: 4
Exiting To-Do List Manager. Goodbye!
```

AI Suggestions:

To run this simple To-Do List Manager, uncomment the last two lines in the code cell above and run it. The `run()` method will start an interactive console where you can add, view, and complete tasks.

Now, let's consider some improvements for error handling and additional features for the To-Do List Manager.

Suggested Improvements:

1. Error Handling:

- **Input Validation:** Ensure user inputs for task numbers are valid integers and within the range of existing tasks.
- **Empty Task Input:** Prevent adding empty tasks.
- **File I/O Errors (if saving/loading):** Handle cases where the application cannot read from or write to a file (e.g., permissions, file not found).

2. Additional Features:

- **Delete Task:** Allow users to remove tasks from the list.
- **Edit Task:** Enable modification of existing task descriptions.
- **Save/Load Tasks:** Persist the to-do list to a file (e.g., CSV, JSON, or a simple text file) so tasks are not lost when the program exits.
- **Prioritization:** Add a priority level to tasks (e.g., high, medium, low).
- **Due Dates:** Allow users to set and view due dates for tasks.
- **Search/Filter Tasks:** Implement functionality to search for tasks by keyword or filter by completion status, priority, or due date.
- **Clear All Completed Tasks:** A quick option to remove all tasks marked as complete.
- **User Interface (GUI):** For a more user-friendly experience, move from a console-based interface to a graphical one (e.g., using `Tkinter`, `PyQt`, `Kivy`).
- **Sorting:** Sort tasks by various criteria (e.g., alphabetical, completion status, due date, priority).

Below is an improved version of the To-Do List Manager focusing on better error handling for user inputs and adding a `delete_task` feature.

Improvised code:

```python
class ImprovedTodoListManager:
    def __init__(self):
        self.tasks = []

    def _get_valid_task_index(self, prompt):
        while True:
            try:
                task_num_str = input(prompt)
                if not task_num_str.strip():
                    print("Input cannot be empty. Please enter a number.")
                    continue
                task_index = int(task_num_str) - 1
                if 0 <= task_index < len(self.tasks):
                    return task_index
                else:
                    print("Invalid task number. Please enter a number from the list.")
            except ValueError:
                print("Invalid input. Please enter a number.")

    def add_task(self, task):
        if not task.strip():
            print("Task description cannot be empty.")
            return
        self.tasks.append({"task": task.strip(), "completed": False})
        print(f"Task '{task.strip()}' added.")

    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
            return
        print("\n--- To-Do List ---")
        for i, task_item in enumerate(self.tasks):
            status = "[X]" if task_item["completed"] else "[ ]"
            print(f"{i + 1}. {status} {task_item["task"]}")
        print("------------------")
```

```python
def complete_task(self):
    self.view_tasks()
    if not self.tasks:
        return
    task_index = self._get_valid_task_index("Enter the task number to complete: ")
    if task_index is not None:
        self.tasks[task_index]["completed"] = True
        print(f"Task '{self.tasks[task_index]["task"]}' marked as complete.")

def delete_task(self):
    self.view_tasks()
    if not self.tasks:
        return
    task_index = self._get_valid_task_index("Enter the task number to delete: ")
    if task_index is not None:
        removed_task = self.tasks.pop(task_index)
        print(f"Task '{removed_task["task"]}' deleted.")

def run(self):
    while True:
        print("\n--- Menu ---")
        print("1. Add Task")
        print("2. View Tasks")
        print("3. Complete Task")
        print("4. Delete Task")
        print("5. Exit")
        print("------------")

        choice = input("Enter your choice: ")

        if choice == '1':
            task = input("Enter the task: ")
            self.add_task(task)
        elif choice == '2':
            self.view_tasks()
        elif choice == '3':
            self.complete_task()
```

```
            choice = input("Enter your choice: ")

            if choice == '1':
                task = input("Enter the task: ")
                self.add_task(task)
            elif choice == '2':
                self.view_tasks()
            elif choice == '3':
                self.complete_task()
            elif choice == '4':
                self.delete_task()
            elif choice == '5':
                print("Exiting To-Do List Manager. Goodbye!")
                break
            else:
                print("Invalid choice. Please try again.")

# To run the improved manager:
improved_todo_manager = ImprovedTodoListManager()
improved_todo_manager.run()
```

Output:

```
Enter the task: add
Task 'add' added.

--- Menu ---
1. Add Task
2. View Tasks
3. Complete Task
4. Delete Task
5. Exit
------------
Enter your choice: 2

--- To-Do List ---
1. [ ] add
------------------

--- Menu ---
1. Add Task
2. View Tasks
3. Complete Task
4. Delete Task
5. Exit
------------
Enter your choice: 4

--- To-Do List ---
1. [ ] add
------------------
Enter the task number to delete: 1
Task 'add' deleted.

--- Menu ---
1. Add Task
2. View Tasks
3. Complete Task
4. Delete Task
5. Exit
------------
Enter your choice: 5
Exiting To-Do List Manager. Goodbye!
```

Explanation:

The initial program provides basic functionality, while AI helps enhance it with better structure, validation, and features. This shows how AI can act as a pair programmer by improving code quality and efficiency.

Task 2 – Version Control Integration

Prompt:

"#Initialize a Git repository, create a branch, add AI-assisted changes, commit them, and merge into the main branch."

```
%%writefile todo_manager_improved.py

class ImprovedTodoListManager:
    def __init__(self):
        self.tasks = []

    def _get_valid_task_index(self, prompt):
        while True:
            try:
                task_num_str = input(prompt)
                if not task_num_str.strip():
                    print("Input cannot be empty. Please enter a number.")
                    continue
                task_index = int(task_num_str) - 1
                if 0 <= task_index < len(self.tasks):
                    return task_index
                else:
                    print("Invalid task number. Please enter a number from the list.")
            except ValueError:
                print("Invalid input. Please enter a number.")

    def add_task(self, task):
        if not task.strip():
            print("Task description cannot be empty.")
            return
        self.tasks.append({"task": task.strip(), "completed": False})
        print(f"Task '{task.strip()}' added.")

    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
            return
        print("\n--- To-Do List ---")
        for i, task_item in enumerate(self.tasks):
            status = "[X]" if task_item["completed"] else "[ ]"
            print(f"{i + 1}. {status} {task_item["task"]}")
```

Code:

```
        print("-----------------")

    def complete_task(self):
        self.view_tasks()
        if not self.tasks:
            return
        task_index = self._get_valid_task_index("Enter the task number to complete: ")
        if task_index is not None:
            self.tasks[task_index]["completed"] = True
            print(f"Task '{self.tasks[task_index]["task"]}' marked as complete.")

    def delete_task(self):
        self.view_tasks()
        if not self.tasks:
            return
        task_index = self._get_valid_task_index("Enter the task number to delete: ")
        if task_index is not None:
            removed_task = self.tasks.pop(task_index)
            print(f"Task '{removed_task["task"]}' deleted.")

    def run(self):
        while True:
            print("\n--- Menu ---")
            print("1. Add Task")
            print("2. View Tasks")
            print("3. Complete Task")
            print("4. Delete Task")
            print("5. Exit")
            print("------------")

            choice = input("Enter your choice: ")

            if choice == '1':
                task = input("Enter the task: ")
                self.add_task(task)
            elif choice == '2':
                self.view_tasks()
```

```
        elif choice == '4':
            self.delete_task()
        elif choice == '5':
            print("Exiting To-Do List Manager. Goodbye!")
            break
        else:
            print("Invalid choice. Please try again.")
```

```
Writing todo_manager_improved.py
```

Step 2: Initialize Git Repository and Make Initial Commit

We will initialize a new Git repository and make an initial commit of our `todo_manager_improved.py` file to the `main` branch.

```
!git init
!git config user.email "colab-user@example.com"
!git config user.name "Colab User"
!git add todo_manager_improved.py
!git commit -m "Initial commit: Added improved To-Do List Manager"
```

```
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /content/.git/
[master (root-commit) f44cde0] Initial commit: Added improved To-Do List Manager
 1 file changed, 81 insertions(+)
 create mode 100644 todo_manager_improved.py
```

Step 3: Create and Switch to a New Branch

We will create a new branch called `ai-enhancements` and switch to it.

```
!git checkout -b ai-enhancements
```

```
Switched to a new branch 'ai-enhancements'
```

AI-Assisted Modifications:

For this step, we'll simulate an AI-assisted modification by making a minor change to our todo_manager_improved.py file, such as adding a comment or a print statement, and then committing it. This represents the 'AI-assisted changes' made on this branch.

Output:

```
%%writefile -a todo_manager_improved.py

# This is a simulated AI-assisted change: added a comment at the end of the file.
```

```
Appending to todo_manager_improved.py
```

```
!git add todo_manager_improved.py
!git commit -m "feat: Add simulated AI-assisted enhancement to todo manager"
```

```
[ai-enhancements f2d0777] feat: Add simulated AI-assisted enhancement to todo manager
 1 file changed, 2 insertions(+)
```

Step 5: Merge Changes into the Main Branch

Now, we'll switch back to the `main` branch and merge the changes from our `ai-enhancements` branch.

```
!git checkout master
!git merge ai-enhancements --no-edit
```

```
Switched to branch 'master'
Updating f44cde0..f2d0777
Fast-forward
 todo_manager_improved.py | 2 ++
 1 file changed, 2 insertions(+)
```

Explanation:

Version control helps track changes, manage different versions, and safely integrate AI-generated code. It enables collaboration by allowing developers to work on separate branches and merge improvements efficiently.

Task 3 – AI for Commit Message Generation

Prompt:

"#Generate meaningful commit messages for code changes and compare them with human-written messages."

Example Changes Made

- Added error handling to To-Do app
- Improved user input validation
- Added delete task feature

Human-Written Commit Messages

- `fixed stuff`
- `updated code`
- `added feature`

Problems:

- Not clear
- No detail
- Hard to understand history

AI-Generated Commit Messages

- `Add exception handling for invalid user input`
- `Implement input validation in To-Do list manager`
- `Add delete task functionality to improve task management`

Advantages:

- Clear and descriptive
- Explains *what* and *why*
- Better for collaboration

Explanation:

AI-generated commit messages are more structured and descriptive compared to vague human-written ones. They improve code documentation and make it easier to track changes in version control systems.

Task 4 – Pair Programming Simulation

Prompt:

"#Simulate pair programming where one developer writes code and another improves it using AI, then integrate using Git."

Student A - Initial Code:

```python
class TodoListManager:
    def __init__(self):
        self.tasks = []

    def add_task(self, task):
        self.tasks.append({"task": task, "completed": False})
        print(f"Task '{task}' added.")

    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
            return
        print("\n--- To-Do List ---")
        for i, task_item in enumerate(self.tasks):
            status = "[X]" if task_item["completed"] else "[ ]"
            print(f"{i + 1}. {status} {task_item["task"]}")
        print("------------------")

    def complete_task(self, task_index):
        if 0 <= task_index < len(self.tasks):
            self.tasks[task_index]["completed"] = True
            print(f"Task '{self.tasks[task_index]["task"]}' marked as complete.")
        else:
            print("Invalid task number.")

    def run(self):
        while True:
            print("\n1. Add Task")
            print("2. View Tasks")
            print("3. Complete Task")
            print("4. Exit")

            choice = input("Enter your choice: ")

            if choice == '1':
                task = input("Enter the task: ")
```

```python
                self.add_task(task)
            elif choice == '2':
                self.view_tasks()
            elif choice == '3':
                self.view_tasks()
                try:
                    task_num = int(input("Enter the task number to complete: ")) - 1
                    self.complete_task(task_num)
                except ValueError:
                    print("Invalid input. Please enter a number.")
            elif choice == '4':
                print("Exiting To-Do List Manager. Goodbye!")
                break
            else:
                print("Invalid choice. Please try again.")
todo_manager = TodoListManager()
todo_manager.run()
```

```
1. Add Task
2. View Tasks
3. Complete Task
4. Exit
Enter your choice: 1
Enter the task: AI Project
Task 'AI Project' added.

1. Add Task
2. View Tasks
3. Complete Task
4. Exit
Enter your choice: 1
Enter the task: AIAC Exam
Task 'AIAC Exam' added.

1. Add Task
2. View Tasks
```

```
1. Add Task
2. View Tasks
3. Complete Task
4. Exit
Enter your choice: 2

--- To-Do List ---
1. [ ] AI Project
2. [ ] AIAC Exam
------------------

1. Add Task
2. View Tasks
3. Complete Task
4. Exit
Enter your choice: 4
Exiting To-Do List Manager. Goodbye!
```

Student B - Improvised Code:

```python
class ImprovedTodoListManager:
    def __init__(self):
        self.tasks = []

    def _get_valid_task_index(self, prompt):
        while True:
            try:
                task_num_str = input(prompt)
                if not task_num_str.strip():
                    print("Input cannot be empty. Please enter a number.")
                    continue
                task_index = int(task_num_str) - 1
                if 0 <= task_index < len(self.tasks):
                    return task_index
                else:
                    print("Invalid task number. Please enter a number from the list.")
            except ValueError:
                print("Invalid input. Please enter a number.")

    def add_task(self, task):
        if not task.strip():
            print("Task description cannot be empty.")
            return
        self.tasks.append({"task": task.strip(), "completed": False})
        print(f"Task '{task.strip()}' added.")

    def view_tasks(self):
        if not self.tasks:
            print("No tasks in the list.")
            return
        print("\n--- To-Do List ---")
        for i, task_item in enumerate(self.tasks):
            status = "[X]" if task_item["completed"] else "[ ]"
            print(f"{i + 1}. {status} {task_item["task"]}")
        print("-----------------")

    def complete_task(self):
```

```python
        self.view_tasks()
        if not self.tasks:
            return
        task_index = self._get_valid_task_index("Enter the task number to complete: ")
        if task_index is not None:
            self.tasks[task_index]["completed"] = True
            print(f"Task '{self.tasks[task_index]["task"]}' marked as complete.")

    def delete_task(self):
        self.view_tasks()
        if not self.tasks:
            return
        task_index = self._get_valid_task_index("Enter the task number to delete: ")
        if task_index is not None:
            removed_task = self.tasks.pop(task_index)
            print(f"Task '{removed_task["task"]}' deleted.")

    def run(self):
        while True:
            print("\n--- Menu ---")
            print("1. Add Task")
            print("2. View Tasks")
            print("3. Complete Task")
            print("4. Delete Task")
            print("5. Exit")
            print("------------")

            choice = input("Enter your choice: ")

            if choice == '1':
                task = input("Enter the task: ")
                self.add_task(task)
            elif choice == '2':
                self.view_tasks()
            elif choice == '3':
                self.complete_task()
            elif choice == '4':
```

```
            elif choice == '4':
                self.delete_task()
            elif choice == '5':
                print("Exiting To-Do List Manager. Goodbye!")
                break
            else:
                print("Invalid choice. Please try again.")
improved_todo_manager = ImprovedTodoListManager()
improved_todo_manager.run()
```

```
--- Menu ---
1. Add Task
2. View Tasks
3. Complete Task
4. Delete Task
5. Exit
------------
Enter your choice: 1
Enter the task: Lab Exam
Task 'Lab Exam' added.

--- Menu ---
1. Add Task
2. View Tasks
3. Complete Task
4. Delete Task
5. Exit
------------
Enter your choice: 1
Enter the task: 2
Task '2' added.

--- Menu ---
1. Add Task
2. View Tasks
3. Complete Task
4. Delete Task
5. Exit
------------
Enter your choice: 4

--- To-Do List ---
1. [ ] Lab Exam
2. [ ] 2
------------------
Enter the task number to delete: 2
Task '2' deleted.

--- Menu ---
1. Add Task
```

```
--- To-Do List ---
1. [ ] Lab Exam
2. [ ] 2
------------------
Enter the task number to delete: 2
Task '2' deleted.

--- Menu ---
1. Add Task
2. View Tasks
3. Complete Task
4. Delete Task
5. Exit
------------
Enter your choice: 2

--- To-Do List ---
1. [ ] Lab Exam
------------------

--- Menu ---
1. Add Task
2. View Tasks
3. Complete Task
4. Delete Task
5. Exit
------------
Enter your choice: 5
Exiting To-Do List Manager. Goodbye!
```

Explanation:

This task demonstrates collaborative development where AI assists in refining code. Using Git ensures smooth integration of contributions, making teamwork more organized and efficient.

Conclusion:

AI can effectively act as a pair programmer by assisting in writing, improving, and documenting code. Integrating AI with version control

systems like Git enhances collaboration and code management. Overall, it improves productivity, code quality, and teamwork in software development.